sat-mb-fidelity___fidelity-kary-f-k4

An AgentMPI run analysis

Abstract

This is the k-ary cell of the saturation attempt: eight ranks, twelve items each, a 450-token merge budget, $k = 4$. The campaign's design intent was to push a semantic reduction past its capacity by tripling the item count and halving the budget relative to the `real-mb-fidelity` baseline, and it failed to do so — retention is 1.000, all 96 identifiers reach the root, none is fabricated. The reason is visible in the store: the operator did not drop and did not overflow its context, it *re-encoded*, inventing a packed notation (`[F-0-0]100n [F-0-1]101d ...`) that carries an item in 14.3 bytes where the input spent 57 to 67. And when re-encoding was not enough it simply exceeded the limit: the root emitted 698 tokens against a stated 450, and the contract stored in this run's fabric records `max_tokens: null` with an empty semantics string, so the budget was a sentence in a prompt rather than a constraint the runtime checked. This cell's own contribution is the cost measurement. Three variadic applications at fold depth $\lceil \log_4 8 \rceil = 2$, against seven applications at depth 3 for the binomial tree and seven at depth 7 for the chain and the flat fold, give the campaign's fastest wall time at 53.07 s, its lowest output-token count at 1,576, and its lowest cost at $0.0369 — with retention held at 1.000. Since this configuration is not capacity-bound, that is the cleanest statement in my whole set of what the k-ary tree buys: fewer operator applications at unchanged quality.

1 What this run is

property	value
run	`sat-mb-fidelity__fidelity-kary-f-k4`
experiment	–
campaign label	–
declared world size	8
ranks appearing in the log	8
executors	`broker` × 8, `worker` × 10
events	63
wall time	53.07 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	1.21×	total busy time over wall time
parallel efficiency	15.1%	achieved parallelism per rank
idle fraction	84.9%	rank-seconds paid for and unused
peak ranks busy	2	of 8 in the world
serial fraction of active time	62.4%	time with exactly one rank busy
load imbalance	5.43×	slowest rank's busy time over the mean
coordination share	17.8%	rank-seconds blocked in collectives, of those available
coordination in flight	100.0%	wall clock with any rank inside a collective
rank reattachments	10	fresh agent processes that took over a rank role
agent calls	3	invocations that reached a model
agent latency p50 / p95	25.51 / 27.31 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	2,913	0 deferred under rendezvous
tokens in / out	4,435 / 1,576	prompt and completion
cost	$0.0369	0.0234 $/kTok out

3 What the analysis notices

- [critical] At least one rank waited 5 s for an agent to claim its task before the broker gave up. That is a pool-sizing failure outside the protocol, not a slow run.
- [note] Parallel efficiency is 15.1%: 84.9% of the rank-seconds paid for went unused.
- [note] Load imbalance is 5.43×: the slowest rank was busy far longer than the mean.
- [ok] 10 rank reattachment(s) occurred, up to incarnation 3: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- [ok] All 1 checkable collective(s) also matched the predicted round count.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	43.52	82.0	2	2	0	4	0	0.0	finalized
1	0.00	0.0	0	0	1	0	398	0.0	finalized
2	0.00	0.0	0	0	1	0	398	0.0	finalized
3	0.00	0.0	0	0	1	0	398	0.0	finalized
4	20.62	74.4	1	1	1	3	525	0.0	finalized
5	0.00	0.0	0	0	1	0	398	0.0	finalized
6	0.00	0.0	0	0	1	0	398	0.0	finalized
7	0.00	0.0	0	0	1	0	398	0.0	finalized

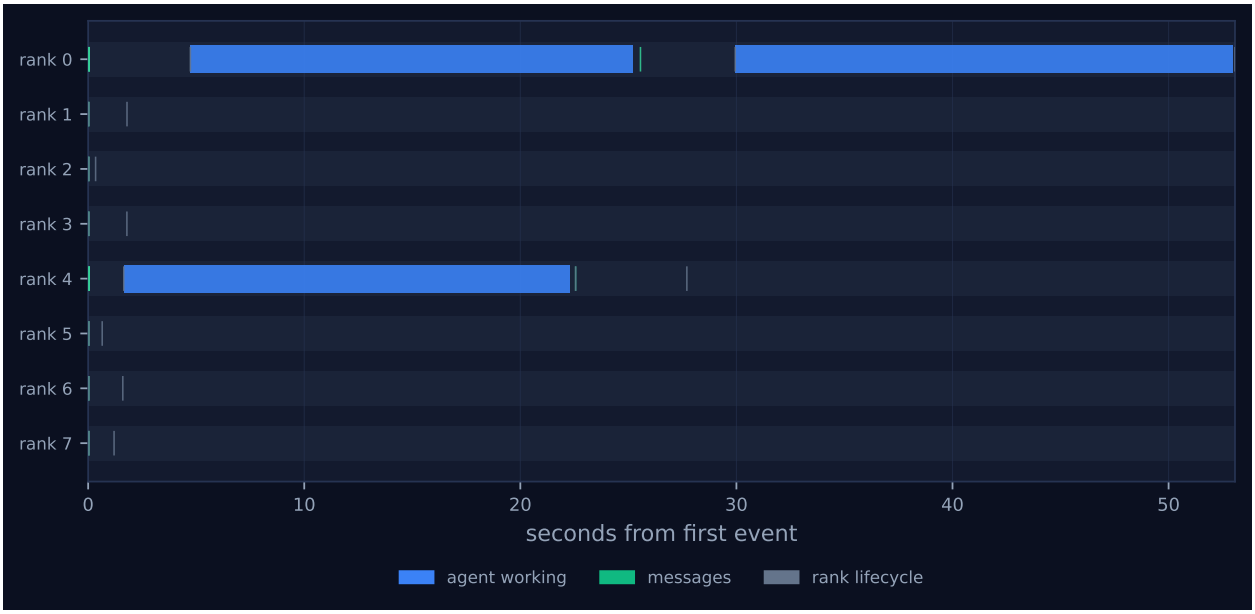

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

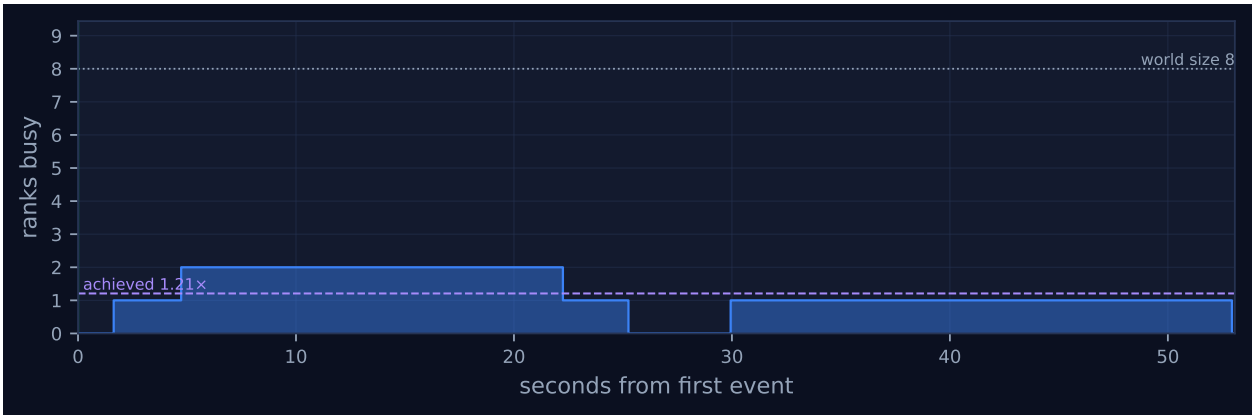

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

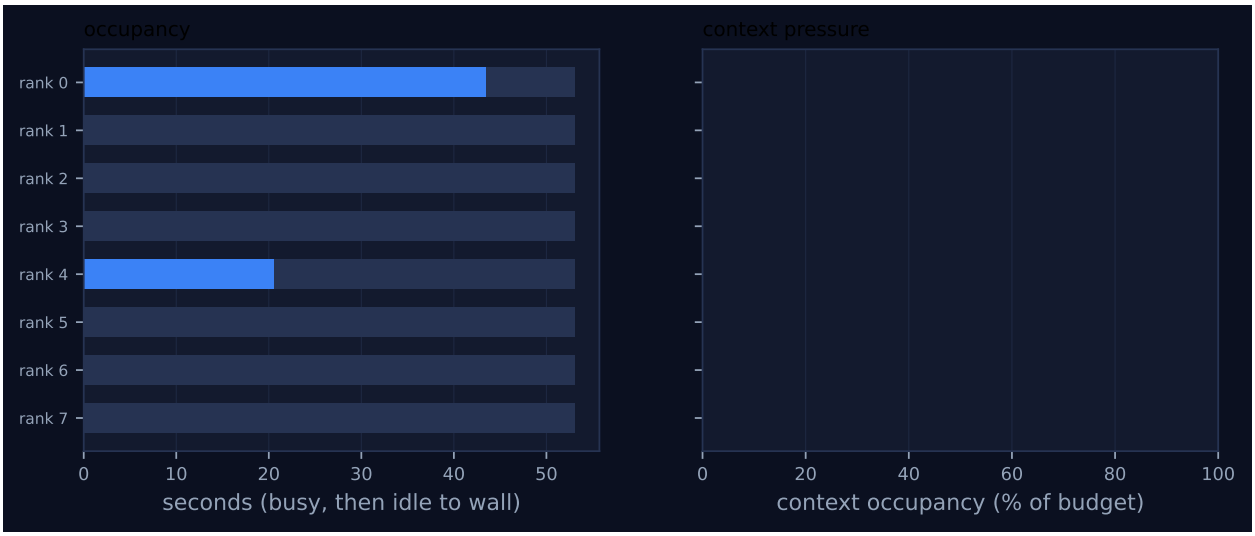

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	kary	–	8	8	2	2	7	7	2,827	53.07	53.04	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 shows two bands and nothing else, which is the k-ary tree in its purest form. Rank 0 and rank 4 hold bars concurrently from about $t = 3\,\text{s}$ to $t = 25\,\text{s}$; rank 0 holds a second bar alone from about $t = 30\,\text{s}$ to 53.07 s. Ranks 1, 2, 3 and 5, 6, 7 each emit a tick inside the first 40 ms and never reappear. The `stride = 1` level puts the ranks with $vr \bmod 4 = 0$ in charge of three children apiece; the `stride = 4` level leaves rank 0 with one child, rank 4. Peak ranks busy is 2 of 8, serial fraction of active time 62.4%, achieved parallelism 1.21×: all three are what a two-level tree whose top level has a single node must produce, and none is a defect. Sixty-three events, no trailing events, and the log's last entry coincides with `job.finish` — this and the `inc-mb` k-ary cell are the only broker-executed runs in my set with no post-completion tail, because they finish before the worker pool starts recycling. The dashboard's health table accordingly shows rank 0 `finalized` rather than the wall of red `fail_stop` suspicions the slower siblings display.

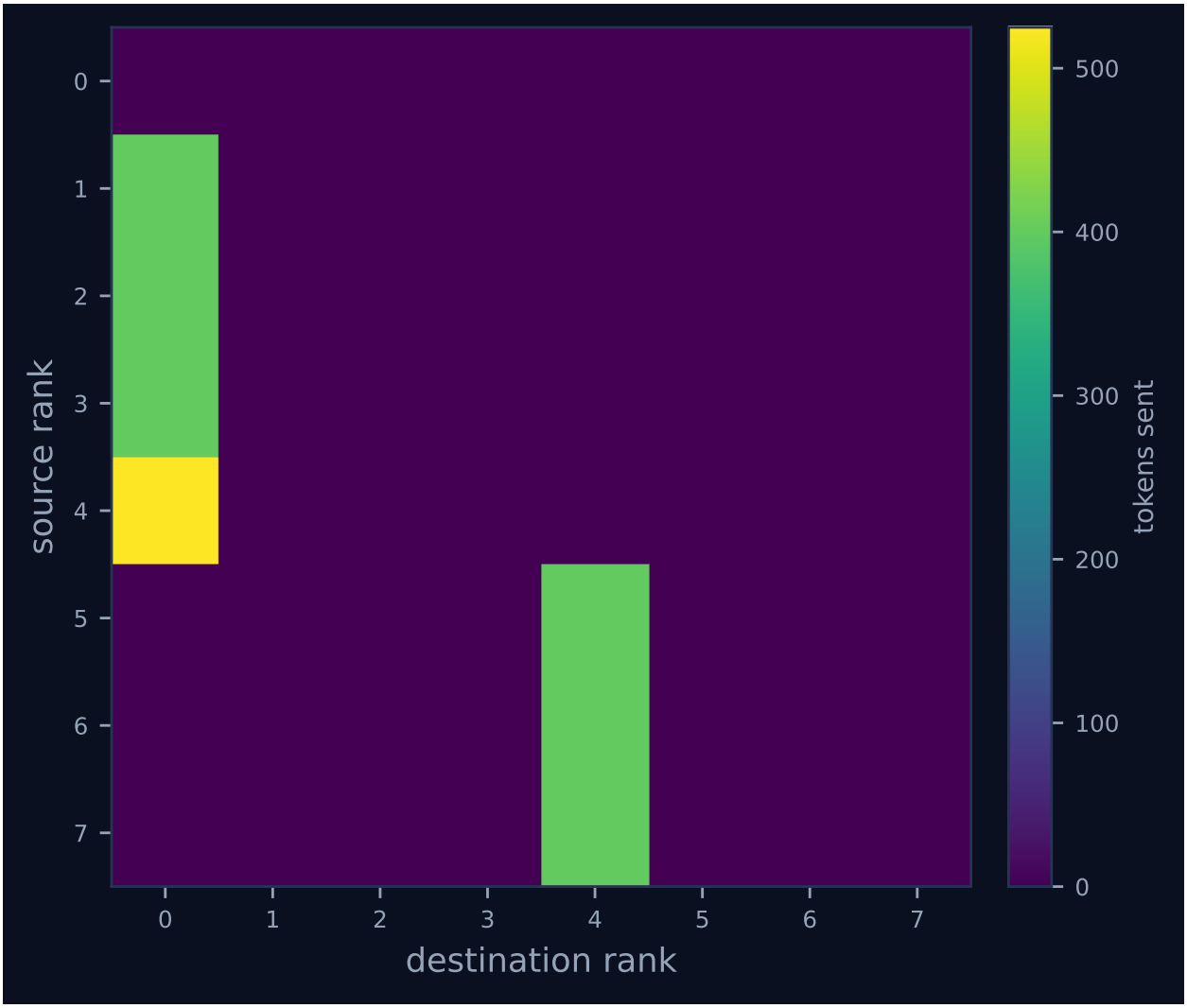

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

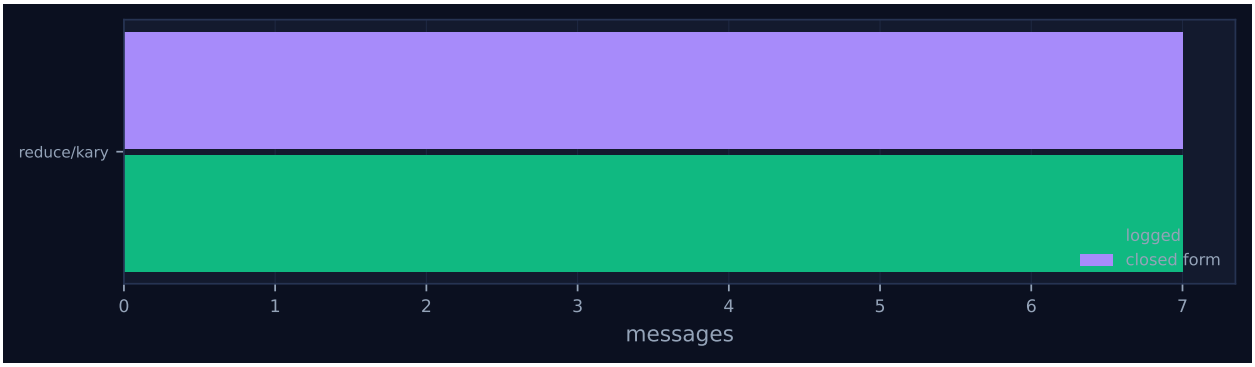

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

Three bars, three latencies: 20.50 s and 20.62 s for the two four-way folds and 23.02 s for the two-way fold above them, on prompts of 1,715, 1,715 and 1,005 tokens. That uniformity is the measurement that matters for the k-ary argument. A level-0 rank here ingests 1,715 prompt tokens — four 398-token reports plus the variadic template — against 933 tokens for a two-way merge in the binomial sibling, and takes the same 20.5 seconds to do it. Doubling the fan-in did not cost latency, which is the empirical content of the claim that an agent rank has no port count, and it is why total prompt consumption is 4,435 tokens here against the binomial cell's 6,318 despite each individual prompt being nearly twice as large. Of the 53.07 s, 64.1 s of generation happened across two concurrent lanes and 10.7 s was spent waiting for a worker to claim a queued call, which I obtained by differencing every `broker.enqueue` → `broker.claim` pair in the log.

The communication matrix, Figure 4, is two columns: $1, 2, 3 \to 0$ and $5, 6, 7 \to 4$ at 398 tokens each, then $4 \to 0$ at 525, seven messages and 2,913 tokens. The message count is $p - 1$, identical to every other algorithm at this p, because widening a reduction tree changes rounds and applications but never messages: every non-root rank sends exactly once. The single 525-token accumulator edge is the whole of this run's non-leaf traffic, against six such edges in the chain.

7 Interpretation

Two things must be established before the retention figure can be read, and the config and the fabric settle both. What the campaign varied: relative to `real-mb-fidelity`, which ran the same $p = 8$ with the same algorithms, this campaign raised `facts` from 4 to 12 and lowered `merge_budget` from 900 to 450, so the root must render 96 items instead of 32 into half the space — a sixfold increase in pressure — and it added `kary` with `fanin: 4`. That is the saturation study, and the target is well defined: the baseline's own accumulators cost about 19.4 tokens per item, so a 450-token budget should hold roughly 23, and 96 items is over four times that. Whether it saturated: no. Retention is 1.000 for all four algorithms, `retention_stdev` is 0.000, `rank_position_correlation` is 0.000, and `make_tables.py` classifies the block empirically as *not capacity-bound*.

The mechanism of the failure to saturate is recoverable from the store and it is the campaign's real result. I extracted every accumulator this run produced and counted bytes per identifier: 25.3 and 16.4 at level 0, and 14.3 at the root, against 57.7 to 67.2 for the depth-1 merges in the binary siblings and 119 for a verbatim input item. The final result is not prose at all but a packed table — `[F-0-0]100n [F-0-1]101d [F-0-2]102s ...` with the workload letters legended in the title — so the operator responded to a tightening budget by inventing a denser encoding, which is exactly what `make_fact_report`'s docstring describes and what motivated the `incompressible` variant. Re-encoding alone was still not sufficient: 96 items at 14.3 bytes is about 7.3 tokens each, or 700 tokens, and the root emitted 698. It got there by ignoring the limit. Every one of the three `agent_calls` rows records `contract` = `{"name": "MergedReport", ..., "max_tokens": null, ..., "semantics": ""}` while `meta` records `{"max_tokens": 450}`: the number went to the executor as a generation hint, which a worker may ignore, and never to the contract checker that would have rejected the overrun and retried. Across this campaign's fifteen merges the outputs run to 698 tokens, 55% over. The run reports zero contract violations, and under a null bound that is uninformative rather than reassuring. The `inc-mb-fidelity-inc` campaign is the corrected experiment: `max_tokens: 450` on the contract, a non-empty semantics clause, an incompressible payload, and retention falls to 0.385 with four of eight ranks erased.

What this particular cell establishes is the cost side, and because quality is pinned at 1.000 it isolates it perfectly. All four algorithms perform the same collective over the same 96 items and send the

same seven messages; this one does it in three operator applications instead of seven, because each variadic call consumes k inputs and retires $k-1$ of them, giving $\lceil (p-1)/(k-1) \rceil = \lceil 7/3 \rceil = 3$. The trace holds exactly three `agent.call` events, labelled `merge:k4:d1` twice and `merge:k2:d2` once so the arity of each is explicit, and the collective records `fold_depth: 2` = $\lceil \log_4 8 \rceil$ and `variadic: true` — the latter worth checking, since `Op.combine` degrades to a left fold when no variadic kernel is attached and a k-ary tree without one spends precisely the depth it was built to save. The result: 53.07 s against 77.56 s for the binomial tree, 197.62 s for the flat fold and 1,376.98 s for the chain; 1,576 output tokens against 2,903, 3,456 and 3,855; $0.0369 against $0.0625, $0.0709 and $0.0782. Since these cells appear in no collective sweep, they and their `inc-mb` counterparts are the only sweep-adjacent evidence for the algorithm, and the argument they support is the cost one rather than the fidelity one. The paper's stated case for k-ary is that quality decays in fold depth so halving the depth buys fidelity; the `inc-mb` campaign refutes that premise by returning byte-identical results at depths 2 and 7. The cost case needs no such premise.

Of the flagged findings, the 15.1% parallel efficiency and the 5.43× imbalance are notes and are structural: a two-level tree over eight ranks can keep at most two of them busy, so six ranks idle by construction. The ten reattachments up to incarnation 3 are the broker's normal mode. The one real problem is in the collectives table, and it is a cost-model defect rather than an implementation one: rounds are 2 against a prediction of 1, and the model column still reads ✓. The tabulated `("reduce", "kary")` entry in `cost.py` evaluates $\lceil \log_k p \rceil$ at `DEFAULT_FANIN = 8` instead of the run's $k = 4$, so it predicts a single 8-way application at depth 1 — a different experiment — and `Analysis.model_checks` counts only `messages_agree`, so the round disagreement is displayed in the table and in `metrics.json` but never reaches the finding text. `predict_kary(p, n, k)` exists and takes an explicit fan-in; `analysis.py` does not use it.

8 Threats to this reading

The executor is real — `executors` records `broker` × 8 and `worker` × 10 — so the three merges were performed by models and the figures above are agent behaviour. The surrogate k-ary cells in `fid-synth` and `fid-inc-smoke` must not be pooled with this one: their `function` executor discards whole accumulator blobs on a draw seeded from the prompt's character length, so they exhibit a loss model rather than testing one.

The most serious threat is that retention 1.000 may not measure information transport at all, and this campaign is where that worry bites hardest. In the compressible configuration `make_fact_report` builds item i of rank r with a throughput of $100+7r+i$ under workload `['nominal','degraded','saturated'][i % 3]`, so the payload is a deterministic function of the identifier. The packed encoding the operator converged on reproduces that rule verbatim: rank 0's twelve items are `100n 101d 102s ...111s` and rank 1's are `107n 108d ...`. A root that had lost an entire message could emit the missing rank's twelve identifiers with correct values by continuing the arithmetic, and `fact_retention` would score it as perfect. Every claim of retention here is a claim about identifier survival on a guessable payload. The `-inc` campaign exists for exactly this reason, scoring the six-digit serial and four-letter checksum as well as the label, and there `n_label_only` is 0 — so the distinction is real and measurable, and it is not measured in this run.

Second, all three of this run's results are byte-identical to results produced in sibling cells, and the four cells ran sequentially against one persistent pool of Cursor subagents. Digest `725368554341` (373 tokens, ranks 0–3) is also the binomial cell's `merge:d2` and the flat cell's `merge:d3`; `56406e77f286` (505 tokens, ranks 4–7) is also the binomial cell's other `merge:d2` and the chain's `merge:d3`; and

`e411144037d6` (698 tokens, all 96) is also the binomial cell's `merge:d3` and the flat cell's `merge:d7`. Three structurally different prompts — a four-way variadic fold, a two-way fold of two subtree accumulators, and a two-way fold of an accumulator with a leaf — returned the same object. I ran the determinism test the contract requires and the executor fails it: prompt digest `20cdeba13af0` appears in both the binomial and chain cells and returns 491 against 490 tokens, and `edf28c1217be` appears in the binomial and flat cells and returns 211 against 354, so identical input does not give identical output and determinism cannot explain the identities. The reading I favour is convergence onto the canonical packed encoding: once the operator commits to `[F-r-i]` value letter and the values follow an arithmetic rule, there is essentially one string that renders a given rank set, which is why the identities cluster on the dense late accumulators and not on the verbose early ones. Cross-run reuse cannot be excluded, and would matter most here, because a remembered 698-token answer would mean this cell's final application never re-derived anything.

Third, one trial per algorithm. The comparison against the binomial tree is safe: 3 applications against 7, 53.07 s against 77.56 s, a ratio of 1.46 where the critical-path count predicts 1.5, and per-call latency differs by 8% between the two runs. The comparison against the chain's 1,376.98 s is not safe at all and should not be quoted: 1,100 s of that run is a worker never showing up, including a single 907.7 s gap between `broker.enqueue` and `broker.claim`, so its generation time is 273.8 s and the 26× ratio is mostly pool starvation. That contamination has escaped into the paper, whose `\NFidChain` macro is filled from this campaign's chain cell.

Finally, this cell's top level is a two-way fold rather than a four-way one, because 8 is not a power of 4, so it measures $k = 4$ at one level and $k = 2$ at the other; the `fid-synth` k-ary cell has the clean geometry at $p = 16$ but is surrogate-executed. `optimal_fanin(32768, 450)` returns its cap of 32, so $k = 4$ was chosen by the `-fanin` flag and not by the policy the specification recommends, and the configuration the policy would pick — one 8-way application at depth 1 — was never run. And the $0.0369 is modelled from $3/Mtok in and $15/Mtok out with `tokenizer_exact: false`, not billed.